

4

4주차 계획: 자료 수합 후 환경 통일

≡ 기한	~ 07/20 (월)
🕒 선택	방캅스
📅 세션 날짜	2026년 7월 14일

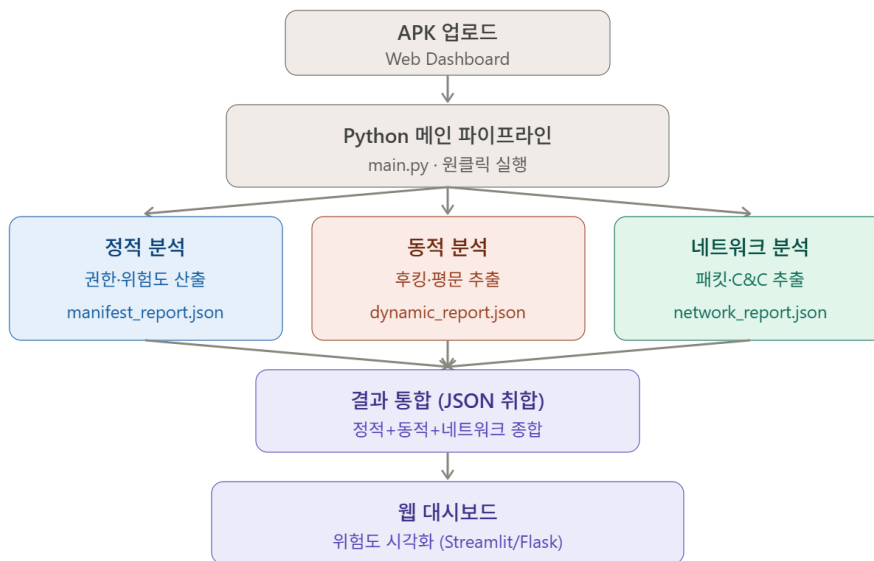
☐ a,b,c,d 모아서 환경 통일하기

일단 있는 a,b,c 에 대해서 자료를 모으고 다른 거 하나로 통일해서 일단 환경 통일부터 하나의 ai에게 자료 싹 모아서 물어보고 그거대로 환경 통일

나온 걸 git에 올려서 그 git을 토대로 하나씩 더 덧붙여서 완성하기

그리고 스키마들이 어떤 역할을 하는지 정리

B. 프로젝트/설계



- JSON 스키마 초안: `schemas/` 이하 위치에 존재

[README.md](#)

[static_report.schema.json](#)

[dynamic_report.schema.json](#)

`network_report.schema.json`

- 아키텍처 다이어그램: `docs/android_xray_architecture.svg`
- 8주차 실제 샘플 테스트 대비 조사: `docs/anubis_공개샘플_조사.md`

`anubis_공개샘플_조사`

아직 팀 논의가 필요한 것

- **GitHub 저장소 연결 / 브랜치 전략:** 팀 git 저장소는 이미 있지만 아직 이 폴더와 연결하지 않은 상태. 브랜치 전략은 팀 전체가 합의해야 하는 부분이라 임의로 정하지 않았음. 아래는 흔히 쓰는 무난한 안이니 참고해서 오늘 회의에서 확정하면 됨:
 - `main`: 항상 동작하는 상태만 유지
 - `dev`: 각자 작업을 합치는 브랜치
 - `feature/static-analyzer`, `feature/dynamic-agent`, `feature/network-analyzer`, `feature/dashboard`: 모듈별 작업 브랜치
- 연결 방법이 결정되면, 이 폴더에서 `git init` → `git remote add origin <팀 저장소 주소>` → `git add . && git commit -m "1주차: 환경 구축 및 아키텍처/스키마 초안"` → `git push` 순서로 진행하면 됨.

정적 분석 모듈 개발 역할 분배

실제 코드가 존재하는 것 (문서에 코드 전문이 있음)

1. `decompiler.py` (B 담당) — 완성

```
run_apktool(apk_path, output_dir, timeout=120)-> Pathrun_jadx(apk_path, output_dir, timeout=120)-> PathDecompileErrorDecompileTimeoutError
```

가짜 실행 파일로 실패 케이스 4종(APK 없음/도구 없음/타임아웃/비정상 종료) 검증까지 완료.

2. `manifest_parser.py` (C 담당) — 완성

```
PERMISSION_WEIGHTS# 디렉터리_get_component_info(manifest_root, tag_name)# 내부 헬퍼parse_manifest(apk_path)->dict
```

androguard 4.1.4 기반, 정상 샘플(`GoogleRestorePrebuilt.apk`)로 실제 실행 검증 완료.

3. `schema.py` (A 담당) — 타입 정의 완성 (실행 코드는 아니지만 확정된 산출물)

```
FileHash, Meta, ManifestInfo, CertInfo, CodeAnalysis, StringsInfo TypedDict들StaticAnalysisResult TypedDict
```

코드가 없는 것 (템플릿/미확인)

파일	상태
<code>analyzer.py</code>	A가 직접 "템플릿 코드"라고 명시 — <code>analyze_static()</code> 이 부르는 6개 함수 모두 미구현 상태로 추정
<code>apk_extractor.py</code>	코드 없음
<code>cert_analyzer.py</code>	코드 없음
<code>code_scanner.py</code>	코드 없음
<code>string_extractor.py</code>	코드 없음

파일	상태
<code>sdk_detector.py</code>	코드 없음
<code>risk_scorer.py</code>	코드 없음 (D 담당 추정, 진행 상황 자체가 공유 안 됨)
<code>exceptions.py</code>	<code>StaticAnalysisError</code> 존재 여부조차 미확인
<code>__init__.py</code>	언급만 있고 내용 없음 (원래 내용 없어도 되는거 아니에여??)

동적 분석 모듈 개발 역할 분배

A: Frida 제어 스크립트

- 3주차: Frida 서버 연결, 세션 생성(attach/spawn), 앱 프로세스 제어 로직 작성
- 4주차: 연결 안정성 보강(재시도, 타임아웃 처리), 여러 앱에 대한 배치 실행 지원
- 산출물: `frida_controller.py`

B: JS 후킹 스크립트

- 3주차: 대상 앱의 주요 클래스/함수 구조 분석, 후킹 대상 함수 목록 정리
- 4주차: `Interceptor.attach` 로 문자열 처리·Base64·XOR·암호화 함수 후킹 코드 작성
- 산출물: `hooks.js`

C: Python-JS 연동 및 데이터 처리

- 3주차: `script.on('message', ...)` 기반 메시지 수신 구조 설계, 데이터 포맷(JSON 스키마) 정의
- 4주차: 후킹 결과 파싱, 필터링(중복 제거·평문 판별), `dynamic_report.json` 생성 로직 작성
- 산출물: 메시지 파서, `dynamic_report.json`

D: 실행 시나리오 자동화

- 3주차: 테스트 시나리오 설계(로그인, 권한 요청 등), ADB/에뮬레이터 기반 앱 실행 자동화
- 4주차: 시나리오별 실행 결과 검증 및 재현성 확보(반복 실행 시 동일 결과 보장), 엡지 케이스 대응(앱 크래시, 미실행 등)
- 산출물: `scenario_runner.py`, 시나리오 정의서

A(제어)와 B(후킹 대상)는 3주차 초반에 후킹 대상 함수 목록을 같이 맞추는 게 좋고, C는 B의 메시지 포맷이 나와야 본격 진행되니 4주차 위주, D는 A의 제어 스크립트가 어느 정도 되어야 실행 테스트가 가능해서 3주차 중반부터 시작하는 걸 추천해요.